ENSy: Design, Implementation, and Application of an Interactive Embedding Navigation System for Neural Language Models

Author: Alexandre da Silva Santos Santos
Institution: Federal University of Maranhão
Program: B.Sc. in Science and Technology – Computer Engineering
Date: August 2026
Version: 1.0.0

Abstract

The internal representations of neural language models, commonly referred to as word embeddings, encode rich semantic and syntactic relationships in high-dimensional vector spaces that resist straightforward human interpretation. This work presents **ENSy (Embedding Navigation System)**, an interactive, browser-based computational environment designed to enable the navigation, exploration, and geometric analysis of high-dimensional semantic spaces. The system integrates a Python-based backend, built with FastAPI and Uvicorn, with a TypeScript and React frontend rendered using the Three.js WebGL engine. Four dimensionality reduction algorithms — UMAP, t-SNE, PCA, and Isomap — project embedding vectors extracted from the Qwen 2.5 0.5B-Instruct language model onto a navigable three-dimensional Cartesian manifold. A central-reticle screen-space raycasting mechanism enables the user to select individual token representations and inspect their five nearest topological neighbors by cosine similarity. Results can be exported as comma-separated value files for downstream statistical analysis. This document provides a comprehensive technical description of the system's architecture, algorithmic foundations, implementation choices, data flow, and user interaction model.

Keywords: word embeddings, embedding navigation, dimensionality reduction, vector space visualization, UMAP, t-SNE, PCA, Isomap, cosine similarity, Three.js, FastAPI, Qwen 2.5, interpretability.

1. Introduction

Modern neural language models (NLMs) rely on embedding matrices to map discrete tokens — linguistic units such as words, subwords, or characters — into continuous, dense vectors in a high-dimensional real-valued space . These embedding vectors serve as the primary substrate through which NLMs encode semantic similarity, syntactic relations, and contextual meaning. The geometric structure of these spaces is not arbitrary: tokens with related meanings tend to occupy proximal regions, forming clusters, sub-manifolds, and directional orientations that encode analogical relationships, such as:

Despite the theoretical richness of these spaces, their high dimensionality — ranging from to dimensions in contemporary models — makes them inherently opaque to direct human inspection. Researchers analyzing model behavior for the purposes of interpretability, bias

detection, or curriculum learning must rely on mathematical abstractions and indirect proxies. This opacity represents a significant bottleneck in the field of explainable artificial intelligence (XAI), where understanding the internal geometry of a model is prerequisite to understanding its behavior.

Dimensionality reduction techniques such as Principal Component Analysis (PCA), t-distributed Stochastic Neighbor Embedding (t-SNE), Uniform Manifold Approximation and Projection (UMAP), and Isomap offer a bridge between the native high-dimensional space and lower-dimensional projections that preserve topological or statistical structure to varying degrees. However, existing tools for applying these techniques to language model embeddings are often limited to static 2D scatter plots, require specialized data science environments, or lack interactive inspection capabilities.

This work addresses these limitations by presenting **ENSy (Embedding Navigation System)**: a full-stack, interactive application that renders the embedding space of the Qwen 2.5 0.5B-Instruct model as a navigable three-dimensional manifold in the browser. The system empowers researchers to interactively explore high-dimensional semantic geometries through real-time first-person flight controls, inspect local topological neighborhoods using screen-space reticle raycasting, contrast four foundational projection algorithms (UMAP, t-SNE, PCA, Isomap) within a unified visual framework, and export neighborhood subsets to standardized CSV format for downstream statistical verification.

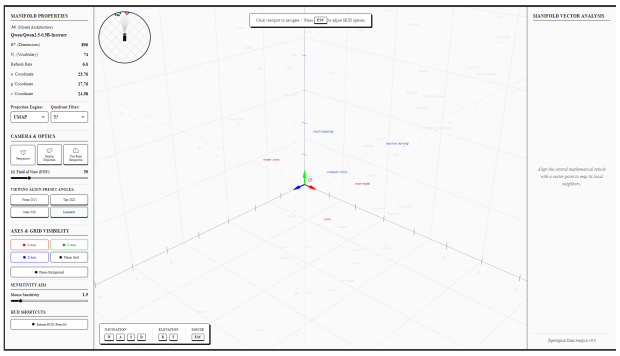


Fig. 1. Complete 1920×1080 workspace in Isometric viewing alignment, illustrating the Left Control Dashboard, central Three.js 3D Manifold Canvas with central crosshair reticle targeting a semantic token, and the populated Right Vector Inspector.

The contribution of this work is both theoretical and practical. It provides a concrete implementation of an XAI instrument for language model geometry, demonstrates the architectural patterns required to couple a machine learning inference pipeline with a real-time 3D rendering engine, and offers an accessible platform for pedagogical exploration of embedding spaces at the undergraduate and graduate levels.

1.1 Research Objectives

The research program guiding this work is articulated through four specific objectives:

O1 — Full-Stack Manifold Rendering. To design and implement an interactive web architecture capable of extracting language model embeddings and rendering them as a navigable 3D Riemannian manifold approximation.

O2 — Multi-Algorithmic Comparative Analysis. To integrate linear (PCA) and non-linear (t-SNE, UMAP, Isomap) dimensionality reduction engines behind a unified asynchronous API, enabling real-time comparative inspection.

O3 — Topological Neighborhood Inspection. To implement a screen-space proximity selection mechanism that dynamically computes and exposes local topological structures via cosine similarity metrics.

O4 — Interpretability and Pedagogical Utility. To demonstrate the system's practical efficacy as an explainable AI tool for interrogating latent cluster organization and geometric semantic relationships.

1.2 Document Organization

This document is organized as follows. Section 2 provides the theoretical background on word embeddings, dimensionality reduction algorithms, and the cosine similarity metric. Section 3 describes the system architecture in full, encompassing the technology stack, component model, and data flow. Section 4 details the backend implementation, including the embedding pipeline and projection endpoints. Section 5 describes the frontend implementation, with emphasis on the Three.js rendering engine, camera control model, and raycasting selection mechanism. Section 6 discusses user interaction design and interface layout. Section 7 presents the data model and static fallback mechanism. Section 8 discusses limitations and directions for future work. Section 9 concludes.

2. Theoretical Background

2.1 Word Embeddings and the Distributional Hypothesis

The mathematical foundation of word embeddings rests on the **distributional hypothesis**, which asserts that words appearing in similar linguistic contexts tend to share semantic properties. Formally, given a vocabulary of tokens, an embedding model defines a mapping:

mapping each token to a dense vector , where . In the specific case of autoregressive transformer models such as Qwen 2.5, this mapping is implemented through a **token embedding matrix** , where each row constitutes the input embedding for token . For Qwen 2.5 0.5B-Instruct, the embedding dimensionality is and the vocabulary size is .

2.2 Cosine Similarity

The **cosine similarity** between two embedding vectors is defined as:

The cosine similarity ranges in the interval , where a value of indicates identical orientation, indicates orthogonality, and indicates opposite orientation. This metric is particularly well-suited for embedding spaces because it is invariant to vector magnitude, focusing exclusively on the angular relationship between vectors. This invariance is crucial because embedding models often produce vectors of varying norms that do not encode semantic information.

In ENSy, cosine similarity is computed for all pairs of tokens using scikit-learn's `cosine_similarity` function on the embedding matrix, yielding a similarity matrix where is the number of active tokens. The five tokens with the highest similarity scores relative to any given query token constitute its **topological neighborhood**.

2.3 Dimensionality Reduction

Since the native embedding dimension cannot be directly visualized, dimensionality reduction is required. ENSy implements four distinct algorithms, each offering different preservation guarantees and structural emphases.

2.3.1 Principal Component Analysis (PCA)

PCA is a classical linear method that projects data onto a lower-dimensional subspace defined by the directions of maximum variance in the

original data. Given a centered data matrix , PCA computes the eigendecomposition of the covariance matrix:

—

selecting the eigenvectors corresponding to the largest eigenvalues as the projection axes. PCA is **globally structure-preserving**: the first principal component captures the maximum variance, and subsequent components capture progressively less variance. However, PCA is a linear transformation and cannot capture nonlinear manifold structure that many high-dimensional datasets exhibit.

In ENSy, PCA is applied independently for 2D and 3D projections using scikit-learn's `PCA(n_components=k, random_state=42)`.

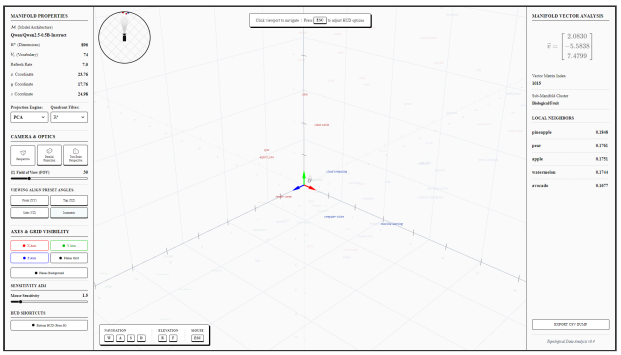


Fig. 2. Linear Principal Component Analysis (PCA) 3D projection of the 896-dimensional embedding space in Isometric perspective.

2.3.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a nonlinear probabilistic method that preserves **local neighborhood structure** in the projection. t-SNE constructs a probability distribution over pairs of points in the high-dimensional space, where the probability reflects how likely points and are to be neighbors under a Gaussian kernel. In the low-dimensional space, a similar distribution is defined using a Student's t-distribution with one degree of freedom. The projection is optimized by minimizing the Kullback-Leibler divergence between and :

—

The **perplexity** hyperparameter controls the effective number of neighbors considered. In the ENSy backend, perplexity is dynamically adapted to the token count as:

—

to prevent instability with small datasets. A key limitation of t-SNE is that it does not preserve global distances: cluster separations in the t-SNE projection are not directly interpretable.

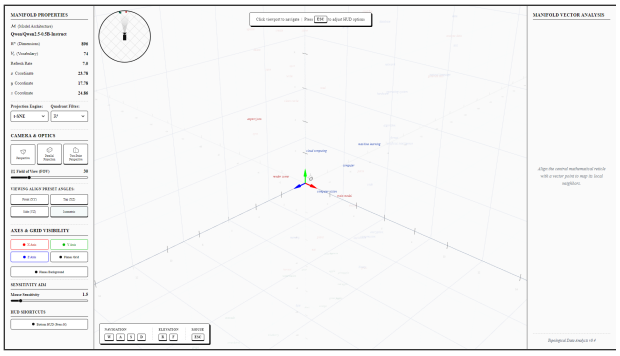


Fig. 3. Non-linear t-SNE 3D projection highlighting localized cluster segregation.

2.3.3 Uniform Manifold Approximation and Projection (UMAP)

UMAP is a manifold learning technique grounded in Riemannian geometry and algebraic topology. UMAP constructs a weighted graph representation of the high-dimensional data, approximating the manifold on which the data lies, then optimizes a low-dimensional layout by minimizing the cross-entropy between the high- and low-dimensional fuzzy topological representations. UMAP preserves both **local and global structure** better than t-SNE and is significantly faster for large datasets.

ENSy applies UMAP with the following parameterization:

The `cosine` metric is specified explicitly, ensuring that UMAP's neighborhood graph is constructed using angular distances consistent with the semantic similarity metric used throughout the system.

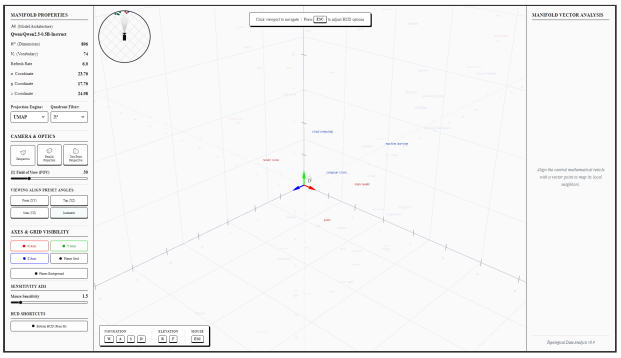


Fig. 4. Default UMAP 3D manifold projection constructed using angular cosine metric.

2.3.4 Isomap

Isomap is a nonlinear method that extends classical multidimensional scaling (MDS) to preserve **geodesic distances** on the data manifold. Isomap first constructs a neighborhood graph with nearest neighbors, then estimates geodesic distances as shortest path lengths in this graph using Dijkstra's algorithm. Finally, classical MDS is applied to the geodesic distance matrix to produce the low-dimensional embedding. Isomap is particularly effective for data lying on a smoothly curved low-dimensional manifold.

In ENSy, Isomap uses and the `cosine` metric for graph construction.

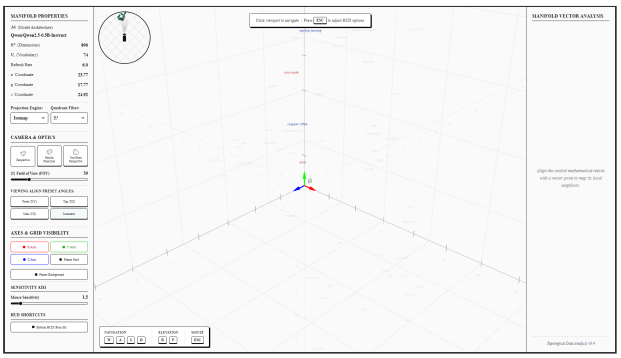


Fig. 5. Non-linear Isomap 3D projection preserving geodesic neighborhood graph distances.

2.4 The Semantic Manifold

The concept of "manifold" in ENSy's nomenclature refers to the mathematical concept of a topological manifold: a space that locally resembles Euclidean space but may possess complex global geometry. The embedding vectors of a language model are hypothesized to lie near a low-dimensional manifold embedded in the high-dimensional vector space . The dimensionality reduction algorithms described above are manifestations of the **manifold hypothesis**: the assumption that high-dimensional data of natural origin concentrates near a low-dimensional manifold, and that this manifold structure encodes meaningful relationships.

3. System Architecture

ENSy follows a two-tier client-server architecture, separating the computationally intensive embedding extraction and dimensionality reduction from the interactive rendering and user interface.

3.1 Technology Stack

ENSy leverages modern web and scientific computing frameworks structured across four primary layers:

Frontend and Graphics Engine. React (v19.0.1) and TypeScript (~v5.8.2) orchestrate component state and static type verification, while Three.js (v0.184.0) manages the WebGL 3D scene graph, shader routines, and canvas rendering. Vite (v6.2.3) serves as the high-throughput development bundler, Tailwind CSS (v4.1.14) and Motion (v12.23.24) govern responsive layout and micro-animations, MathJax (v3) provides client-side LaTeX equation rendering, and Lucide React (v0.546.0) delivers vector iconography.

Backend and Asynchronous Serving. High-throughput REST API serving is provided by FastAPI (v0.115.8) running on the asynchronous

Uvicorn (v0.34.0) ASGI server.

Machine Learning and Reduction Pipeline. Token embedding extraction is performed on Qwen 2.5 0.5B-Instruct using PyTorch (v2.6.0) and HuggingFace Transformers (v4.49.0). Dimensionality reduction algorithms (PCA, t-SNE, Isomap, UMAP) are executed via scikit-learn (v1.6.1) and UMAP (v0.5.7), supported by vectorized NumPy (v2.1.3) linear algebra.

Verification and Quality Assurance. Automated endpoint validation and visual regression testing are conducted using Pytest (v8.3.5) and headless Playwright (v1.48.0).

3.2 Data Flow

The system's data flow proceeds through the following stages:

Stage 1 — Application Initialization. Upon server startup (`app.py`), the dependency installer verifies and installs missing Python packages. The bootstrap vector set ( ) is generated using a seeded pseudo-random normal distribution and normalized to unit length. A daemon thread is immediately launched to load the Qwen 2.5 model in the background.

Stage 2 — Model Loading (Background). The background thread loads the Qwen 2.5 tokenizer and model via the HuggingFace Transformers library. The full embedding matrix is extracted from `model.get_input_embeddings().weight`. Each target word is validated: words that tokenize to multiple subword tokens are excluded from the active set (denoted as `dropped_tokens`). Upon successful loading, the global `active_vectors` and `active_words` pointers are atomically updated.

Stage 3 — API Request. The React frontend issues an HTTP GET request to `/api/projection/{engine_type}` whenever the user changes the projection algorithm. The Vite development server proxies this request to the FastAPI backend at `http://127.0.0.1:3000`.

Stage 4 — Projection and Similarity Computation. The backend applies the requested dimensionality reduction algorithm to `active_vectors`, producing 2D coordinates ( ) and 3D coordinates ( ) for each token. It then computes the cosine similarity matrix and extracts the top-5 neighbors for each token.

Stage 5 — JSON Response. The backend returns a JSON object conforming to the `SemanticSpace` interface. The frontend's `App.tsx` stores this in the `semanticSpace` state variable.

Stage 6 — 3D Scene Update. `SpaceVisualizer` detects the updated data and calls `syncProjectionDataPrivate()`, which linearly interpolates each token's 3D point towards its new projected position over subsequent animation frames.

Stage 7 — Offline Fallback. If the API request fails, `App.tsx` catches the exception and loads the static `semanticSpaceData` from `src/data/semanticSpace.ts`, ensuring the application remains fully functional at all times.

4. Backend Implementation

4.1 Server Entrypoint and Dependency Management

The backend server (`app.py`) begins with a self-bootstrapping dependency management routine (`install_deps()`). Before importing any scientific computing library, the function iterates over a required package list (`fastapi`, `uvicorn`, `transformers`, `torch`, `scikit-learn`, `umap-learn`) and attempts an import for each. Any package that fails the import test is installed via `subprocess.check_call([sys.executable, "-m", "pip", "install", ...])`. This strategy eliminates the need for a pre-configured environment.

4.2 Data Configuration: Cluster Taxonomy and Token Registry

The backend defines a curated vocabulary of 36 tokens organized into three semantic clusters: the **Technology/File** cluster (`#378ADD`), covering computing concepts such as `document`, `file`, `hardware`, `memory`, `binary`, `database`, and `network`; the **Biological/Fruit** cluster (`#1D9E75`), encompassing natural entities including `apple`, `banana`, `orange`, `fruit`, `tree`, and `seed`; and the **Action/Verb** cluster (`#D85A30`), representing procedural semantics like `write`, `read`, `parse`, `encode`, `plant`, `compile`, and `analyze`.

Each cluster is assigned a unique hexadecimal color code (`#378ADD`, `#1D9E75`, `#D85A30`) used consistently across both the 3D scene and the minimap. A `token_registry` dictionary maps each word to its token ID and cluster membership, enabling metadata lookup during JSON serialization.

4.3 Bootstrap Vector Initialization

To guarantee zero-latency API responses before the language model finishes loading, the backend initializes a set of **bootstrap vectors** via seeded pseudo-random normal sampling with unit normalization (see Appendix D, Listing D.1).

The use of a fixed random seed ( ) ensures reproducibility across server restarts. normalization to unit length is necessary because the cosine similarity metric operates on the angular component of vectors; unnormalized bootstrap vectors would exhibit arbitrary magnitude variation that artificially inflates or deflates similarity scores.

4.4 Asynchronous Model Loading

The model loading procedure is executed asynchronously via a Python daemon thread spawned upon server startup (see Appendix D, Listing D.2).

The background initialization sequence executes five sequential stages:

Tokenizer and Architecture Instantiation. The pipeline instantiates the HuggingFace `AutoTokenizer` and `AutoModelForCausalLM` using half-precision (`float16`) to minimize memory footprint on host CPU execution.

Embedding Matrix Extraction. The input embedding weights are decoupled from the PyTorch computation graph via `get_input_embeddings().weight` and cast to float32 NumPy arrays.

Single-Token Validation. Each candidate vocabulary term is evaluated with the tokenizer. Words decomposing into multiple subwords are registered into `dropped_tokens` and excluded, guaranteeing that visualized points represent atomic semantic tokens.

Atomic Pointer Replacement. Following validation, the global references `active_words`, `active_vectors`, and `token_registry` are updated simultaneously. Python's Global Interpreter Lock (GIL) ensures atomic reference reassignment, maintaining thread safety without locking overhead.

4.5 Projection Endpoint

The core API endpoint is `GET /api/projection/{engine_type}`, accepting `umap`, `pca`, `tsne`, or `isomap`.

For each algorithm, the backend instantiates both a 2D and a 3D reducer, applies `fit_transform(vectors)`, and computes the full cosine similarity matrix. Adaptive safeguards are applied dynamically to guarantee algorithmic stability with finite token sets: UMAP enforces ; t-SNE bounds perplexity via ; and Isomap constrains graph connectivity with .

The endpoint packages the projected 2D and 3D coordinates, cluster labels, and top-5 cosine neighbors for each node into a serialized response conforming to the `SemanticSpace` data contract (see Appendix B and Appendix D, Listing D.3).

4.6 Static File Serving and SPA Routing

In production mode, FastAPI mounts the `dist/` directory and serves the React SPA. The fallback route `GET /{rest_of_path:path}` returns `index.html` for any non-API path, enabling client-side React navigation.

5. Frontend Implementation

5.1 Component Architecture

The frontend architecture is partitioned across four specialized React components coordinated by the root `App.tsx`. The root orchestrator maintains global application state (`selectedEngine`, `selectedNode`, `semanticSpace`), handles HTTP communication with the backend API, and manages fallback routing. `SidebarLeft.tsx` hosts algorithm toggles, model metadata badges, and server connectivity monitors. `SpaceVisualizer.tsx` encapsulates the Three.js WebGL canvas, 6-DOF camera mechanics, and HUD overlays. `SidebarRight.tsx` inspects node coordinates, presents top-5 cosine neighbors, and provides CSV exports. `BottomHUD.tsx` renders non-intrusive navigation indicators.

All inter-component communication occurs through unidirectional props and callbacks, preserving deterministic state propagation via `useState` and `useCallback` hooks.

5.1.1 State Persistence

Application settings are debounced and serialized to `localStorage` at one-second intervals to prevent frame drops during the 60 FPS animation loop. Persisted attributes include the active reduction engine (`projectionType`), octant spatial filter (`currentFilter`), camera projection geometry (`cameraMode`), field of view (`fov`), coordinate axis visibility flags (`showXAxis`, `showYAxis`, `showZAxis`), reference grid planes, mouse sensitivity coefficients, and 6-DOF camera coordinates.

5.2 Three.js Scene Construction

The `SpaceVisualizer` component initializes the Three.js rendering pipeline within an empty-dependency `useEffect` hook, isolating the WebGL scene graph from React reconciliation cycles.

5.2.1 Scene Graph

The Three.js scene graph consists of a primary `THREE.Scene` configured with exponential atmospheric fog ( ), an ambient illumination source (`THREE.AmbientLight(0xffffff, 1)`), a composite `gridGroup` containing reference planes, axis arrows, and tick markers, and a collection of zero-opacity `THREE.Points` single-vertex proxies serving as raycasting targets.

5.2.2 Dual Camera System

Three distinct camera modes require both a perspective and an orthographic camera:

Perspective mode (`THREE.PerspectiveCamera`): standard projection model; FOV is user-configurable from to .

Parallel (Orthographic) mode (`THREE.OrthographicCamera`): orthographic projection for engineering views. Bounds are dynamically synchronized to the perspective camera's current frustum:

—— ——

Two-point perspective mode: approximation of architectural two-point perspective, implemented by injecting a vertical shear into the projection matrix:

```
persp.projectionMatrix.elements[9]                =
Math.tan(persp.rotation.x);
```

5.3 Camera Controller: 6-DOF Flight Mechanics

ENSy implements a **six-degrees-of-freedom (6-DOF) first-person flight control scheme** using the browser's Pointer Lock API. When the

user clicks the viewport, `requestPointerLock()` captures the mouse cursor.

Mouse movement updates camera orientation in `YXZ` Euler rotation order:

_ _

where is the default sensitivity and the pitch clamp prevents gimbal lock. Mouse movement is clamped to pixels per event.

Keyboard translation is computed each frame with time-delta scaling:

where and is capped at . The forward direction is projected onto the XZ plane to ensure horizontal movement regardless of pitch.

5.4 Camera Preset Transitions (LERP/SLERP)

Camera preset transitions use a **damped interpolation** scheme per-frame:

Position LERP (Linear Interpolation):

Quaternion-based SLERP is used for rotation to prevent gimbal lock. Interpolation terminates when position distance and rotation angle .

Preset camera transitions align the viewport to canonical orthogonal and perspective projections: **Front** view targets position with rotation ; **Top** view targets with rotation ; **Side** view targets with rotation ; and **Isometric** view targets with pitch and yaw .

5.5 Token Rendering and Label Overlay System

Token labels are rendered through a hybrid CSS/WebGL overlay architecture. An invisible `THREE.Points` object per token acts as a 3D spatial anchor and raycasting target, while a corresponding `<div>` HTML element is positioned absolutely over the WebGL canvas. Each animation frame, 3D world coordinates are projected into 2D Normalized Device Coordinates (NDC) and converted to screen pixels:

Visual clarity is maintained through dynamic depth sorting, where guarantees that foreground labels correctly occlude background points. Label transparency is governed by a distance attenuation function:

Additionally, labels positioned behind reference planes have opacity attenuated by a factor of . Visual clutter in dense clusters is mitigated through an 8-pixel spatial binning system that radially spreads conflicting labels. Algorithm-specific visual enhancements are applied dynamically: PCA incorporates a rotational offset with Z-axis amplification; t-SNE applies a signed power-law compression ; and Isomap enforces polar-to-Cartesian radial spacing.

5.6 Node Selection by Central Reticle

ENSy implements a screen-space reticle selection mechanism evaluated on every animation frame. The viewport center is established as

. For each candidate point verified to lie in front of the camera plane ( ), the 2D Euclidean screen distance is computed:

The token that minimizes within an activation radius of is selected, triggering the `onNodeHovered` state transition and highlighting the corresponding label with the `.highlighted-acad` CSS class. Screen-space proximity selection is substantially more robust than Three.js geometric raycasting, as single-vertex point proxies lack surface area and suffer from floating-point collision failures.

5.7 Token Position Interpolation on Projection Switch

When switching projection algorithms, new coordinates arrive from the API asynchronously. The `syncProjectionDataPrivate()` function updates each token's `targetPosition`, and the animation loop smoothly interpolates:

This LERP factor produces a transition of approximately – frames (– at), allowing the user to observe the topological reorganization of the embedding space.

5.8 Adaptive Grid Resolution

Grid tick density dynamically adapts to camera distance from the origin to maintain visual clarity: intervals are configured as for , for , for , for (default scale), and for .

5.9 Minimap

A 2D circular minimap is rendered each frame on a `<canvas>` element overlaid on the viewport. It depicts all token positions projected onto the XZ plane, colored by cluster, with a camera orientation indicator.

6. User Interface Design

6.1 Layout Philosophy

The interface adopts a three-panel editorial layout designed for scholarly rigor: a 280-pixel Left Sidebar hosting engine controls and model metrics; a central WebGL viewport for full-fidelity 3D exploration; and a 280-pixel Right Sidebar functioning as a topological vector inspector. Typography is strictly structured using **EB Garamond** for editorial prose, **Fira Code** for monospaced metrics, and **MathJax 3** for mathematical expressions.

6.2 Left Sidebar: Control Dashboard

The control dashboard organizes administrative controls across four functional groups:

Manifold Properties. Displays active model architecture ( ), embedding dimensionality ( ), vocabulary scale ( ), and real-time animation FPS performance.

Projection Engine. Provides interactive switching between UMAP, t-SNE, PCA, and Isomap algorithms, accompanied by spatial octant filters (Positive, Negative, or Full volume).

Camera Optics. Hosts projection mode selectors (Perspective, Parallel orthographic, Two-point perspective), field-of-view sliders (–), and direct preset triggers (Front, Top, Side, Isometric).

Cartesian Apparatus. Allows fine-grained toggling of coordinate axes (X, Y, Z), reference plane grids, mouse navigation sensitivity, and HUD indicators.

6.3 Right Sidebar: Vector Inspector

Upon selecting a token, the inspector renders the token's coordinate vector:

alongside its global vocabulary matrix index, semantic cluster badge, and a ranked table of the top-5 nearest neighbors by cosine similarity (evaluated to four decimal precision). A dedicated export trigger facilitates downloading neighborhood data as comma-separated values (`vector_{label}_neighbors.csv`).

6.4 Bottom Navigation HUD

A minimal floating overlay monitors real-time keypress states (W, A, S, D, R, F, ESC) with instantaneous typographic highlights, providing immediate visual feedback during flight navigation.

7. Data Model

7.1 TypeScript Interfaces

The frontend data model is structured around a set of TypeScript interfaces (`Coords3D`, `Neighbor`, `SemanticNode`, `SemanticSpace`) defined in `src/types.ts` that enforce compile-time verification across coordinate payloads, similarity scores, and model metadata (see Appendix D, Listing D.4).

Both `id` (sequential index) and `token_id` (vocabulary index) fields address the discrepancy between sequential node ordering and actual model vocabulary identifiers.

7.2 Static Fallback Dataset

`src/data/semanticSpace.ts` contains 74 pre-computed semantic nodes with UMAP-projected coordinates derived from actual Qwen 2.5 0.5B-Instruct execution (, ,). This ensures a meaningful experience even without the Python backend.

8. Limitations and Future Work

8.1 Current Limitations

L1 — Vocabulary size. 36 tokens in 3 clusters is insufficient to capture the semantic diversity of the Qwen 2.5 embedding space. Scaling to thousands of tokens requires batched similarity computation and instanced mesh rendering ( ).

L2 — Single model support. The backend is hard-coded to Qwen 2.5 0.5B-Instruct. Future work should parameterize the model identifier via API endpoint or environment variable.

L3 — Context-free embeddings. Static input embeddings do not incorporate contextual information from attention layers. Contextualized representations (hidden states from intermediate layers) would require a more complex inference pipeline.

L4 — Viewport projection mode toggle. While 2D planar coordinates ( ) are computed alongside 3D coordinates and leveraged for the tactical minimap and export payloads, the primary Three.js viewport operates in 3D flight mode. A dedicated toggle between full-screen 2D planar scatter views and 3D manifold flight would provide alternative analytical perspectives.

L5 — Single-machine deployment. Production deployment would require containerization (Docker) and production ASGI server configuration.

8.2 Future Work

F1 — Contextual embeddings. Sentence-level hidden state extraction would enable visualization of how token representations shift across contexts.

F2 — Semantic search. A text input for highlighting nearest neighbors across all tokens would accelerate exploratory analysis.

F3 — Trajectory visualization. Rendering interpolation paths between projection algorithms as visible trajectories would reveal embedding stability.

F4 — Multi-model comparison. Rendering two embedding spaces simultaneously would enable direct geometric comparison across architectures.

F5 — Quantitative metrics. Integration of silhouette coefficient, Davies-Bouldin index, and Calinski-Harabász index would provide quantitative cluster validation.

9. Conclusion

This work presented **ENSy (Embedding Navigation System)**, a full-stack interactive system for the three-dimensional navigation, exploration, and analysis of neural language model embedding spaces. The system integrates a FastAPI backend that extracts token embeddings from Qwen 2.5 0.5B-Instruct and applies UMAP, t-SNE, PCA, and Isomap dimensionality reduction, with a React and Three.js frontend rendering the resulting projections as a navigable first-person 3D manifold.

ENSy addresses a genuine gap in the explainable AI tooling landscape: the absence of accessible, interactive, browser-native instruments for actively navigating and analyzing the geometric structure of language model embedding spaces. By coupling a Python machine learning pipeline with a WebGL rendering engine through a clean REST API, the system establishes a scalable paradigm for interactive AI interpretability platforms.

The system's design priorities — zero-dependency frontend fallback, bootstrap vector initialization for instant startup, adaptive grid resolution, damped interpolation for smooth algorithm transitions, and publication-quality mathematical typography — reflect a commitment to both scientific rigor and user experience quality.

Appendix A: File Structure Reference

```
manifold/
├── app.py                          FastAPI backend server (298 lines)
├── requirements.txt                Unified Python dependencies (backend + testing)
├── index.html                      HTML template; MathJax configuration
├── package.json                    Node.js package manifest
├── vite.config.ts                  Vite build configuration; API proxy
├── tsconfig.json                   TypeScript compiler options
└── src/
    ├── main.tsx                    React application entrypoint
    ├── App.tsx                     Root component; state orchestration (264 lines)
    ├── types.ts                    TypeScript data model interfaces (40 lines)
    ├── index.css                   Global CSS styles
    ├── components/
    │   ├── SpaceVisualizer.tsx     Three.js renderer; flight controls (2,243 lines)
    │   ├── SidebarLeft.tsx         Control dashboard (406 lines)
    │   ├── SidebarRight.tsx        Vector inspector (134 lines)
    │   └── BottomHUD.tsx           Navigation indicator (86 lines)
    └── data/
        └── semanticSpace.ts        Static fallback dataset (74 nodes, 2,718 lines)
```

Appendix B: API Reference

`GET /api/projection/{engine_type}`

Path parameter: `engine_type` in {`umap`, `pca`, `tsne`, `isomap`}

Response schema:

```
{
  status: "STABLE" | "LOADING",
  meta: {
    model: string,
    embedding_dim: number,
    total_tokens: number,
    dropped_tokens: string[],
    umap_params?:   { n_neighbors: number, min_dist: number, metric: string },
    pca_params?:    { n_components: number, svd_solver: string },
    tsne_params?:   { perplexity: number, metric: string, init: string },
    isomap_params?: { n_neighbors: number, metric: string }
  },
  nodes: Array<{
    id: number,
    label: string,
    token_id: number,
    cluster: string,
    color: string,
    coords: {
      x2d: number, y2d: number,
      x3d: number, y3d: number, z3d: number
    },
    top5_neighbors: Array<{
      label: string,
      cosine_similarity: number
    }>
  }>
}
```

Appendix C: Keyboard Controls Reference

Input Binding	Subsystem / Operation	Functional Description
Click Viewport	Input Management	Engages browser pointer lock mode (`requestPointerLock()`) to capture mouse orientation.
`W` / `S`	6-DOF Flight Camera	Translates the camera position forward or backward along the horizontal XZ world plane.
`A` / `D`	6-DOF Flight Camera	Strafes the camera position left or right perpendicular to the current gaze direction vector.
`R` / `F`	6-DOF Flight Camera	Translates the camera position vertically along the global Cartesian Y-axis (ascend / descend).
Mouse Motion	6-DOF Flight Camera	Adjusts camera orientation (pitch and yaw) with clamped pitch angle to eliminate gimbal lock.
Scroll Wheel	Optical Controls	Adjusts focal distance zoom (translates along the local Z-axis in perspective mode; scales orthographic frustum).
`H`	User Interface	Toggles visibility of the bottom flight navigation HUD overlay.
`P`	Developer Diagnostics	Outputs point cloud memory metrics, raycasting statistics, and vector state to the browser console.
`Escape`	Input Management	Releases browser pointer lock and restores standard cursor interaction mode.

Appendix D: Implementation Listings

Listing D.1 — Bootstrap Vector Initialization (`app.py`)

```
bootstrap_vectors = np.random.RandomState(42).normal(0, 1, (len(all_words), 896))
bootstrap_vectors /= np.linalg.norm(bootstrap_vectors, axis=1, keepdims=True)
```

Listing D.2 — Asynchronous Model Loading (`app.py`)

```
@app.on_event("startup")
def startup_event():
    threading.Thread(target=load_model_background, daemon=True).start()
```

Listing D.3 — Node Schema Serialization (`app.py`)

```
{
  "id": 0,
  "label": "document",
  "token_id": 6062,
  "cluster": "Technology/File",
  "color": "#378ADD",
  "coords": {
    "x2d": 9.547721,
    "y2d": 7.391323,
    "x3d": 5.423237,
    "y3d": 2.238469,
    "z3d": 6.157989
  },
  "top5_neighbors": [
    { "label": "scan document", "cosine_similarity": 0.645582 }
  ]
}
```

Listing D.4 — TypeScript Data Model Interfaces (`src/types.ts`)

```
interface Coords3D {
  x2d: number;
  y2d: number;
  x3d: number;
  y3d: number;
  z3d: number;
}

interface Neighbor {
  label: string;
  cosine_similarity: number;
}

interface SemanticNode {
  id?: number;
  token_id: number;
  label: string;
  cluster: string;
  color: string;
  coords: Coords3D;
  top5_neighbors: Neighbor[];
}

interface SemanticSpace {
  meta: SemanticMeta;
  nodes: SemanticNode[];
}
```